

Ijin Yu

Introduction In this notebook, we'll implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage. You will use CPU for this notebook.

SSM Update Rule We consider an Linear RNN described by the update: $h_{t+1} = W h_t + U x_t + b$ for $t = 0, 1, \dots, T - 1$. The variables are: - $h_t \in \mathbb{R}^H$, the hidden state at time t . - $x_t \in \mathbb{R}^{N \times D}$, the input at time t . - $W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix. - $U \in \mathbb{R}^{H \times D}$, the input projection matrix. - $b \in \mathbb{R}^H$, the bias vector. N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H . Below you will implement the forward pass for the SSM using recursion based approach. The `unrolled\_ssm\_forward` function will take weights W , U , b and input x and return the hidden states h across different time steps.

```
def unrolled_ssm_forward(W, U, b, x): """ Unroll the linear RNN in time:  $h_{t+1} = W h_t + U x_t + b$  with initial  $h_0 = 0$ . Args: W: (H, H) weight matrix U: (H, D) input projection b: (H,) bias x: (N, T, D) input sequence over T steps Returns: h_all: (N, T, H) hidden states for  $t=1..T$  ( $h\_all[t]$  corresponds to  $h_{t+1}$  in the usual notation). """#####  
## # TODO: Implement the recurrent pass here #  
#####  
##  
#####  
## # END OF YOUR CODE #  
#####  
## return h_all
```

Claude Opus

SSM Forward Pass Implementation

Let me implement the unrolled (recursive) forward pass for the State Space Model.

```
python  
def unrolled_ssm_forward(W, U, b, x):  
    """  
    Unroll the linear RNN in time:  
         $h_{t+1} = W h_t + U x_t + b$   
    with initial  $h_0 = 0$ .  
  
    Args:
```

```

W: (H, H) weight matrix
U: (H, D) input projection
b: (H,) bias
x: (N, T, D) input sequence over T steps
Returns:
    h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
    ""

#####
#####
#                               TODO: Implement the recurrent pass here
#

#####
#####

N, T, D = x.shape
H = W.shape[0]

# Initialize output tensor to store all hidden states
h_all = np.zeros((N, T, H))

# Initialize hidden state to zeros
h_t = np.zeros((N, H))

# Unroll through time
for t in range(T):
    # h_{t+1} = W @ h_t + U @ x_t + b
    # For batched computation:
    # h_t: (N, H), W: (H, H) -> h_t @ W.T gives (N, H)
    # x[:, t, :]: (N, D), U: (H, D) -> x[:, t, :] @ U.T gives (N,
H)
    h_t = h_t @ W.T + x[:, t, :] @ U.T + b
    h_all[:, t, :] = h_t

```

```
#####
#####
#                               END OF YOUR CODE
#

#####
#####
    return h_all
```

Explanation

The implementation follows the recurrence relation $h_{t+1} = W h_t + U x_t + b_{t+1} = W h_t + U x_t + b_{t+1}$:

1. **Extract dimensions:** Get batch size N , sequence length T , and input dimension D from the input tensor. Hidden dimension H comes from the weight matrix W .
2. **Initialize storage:** Create `h_all` of shape (N, T, H) to store all hidden states, and initialize `h_t` to zeros (since $h_0 = 0$).
3. **Iterate through time:** For each timestep t :
 - Compute the new hidden state using the update rule
 - The matrix multiplications are done with transposed weights to handle the batch dimension correctly:
 - `h_t @ W.T` computes $W h_t$ for all batch elements
 - `x[:, t, :] @ U.T` computes $U x_t$ for all batch elements
 - Store the result in `h_all[:, t, :]`

This recursive approach has $O(T)O(T)O(T)$ sequential steps, which is the main limitation compared to the convolution-based approach you'll likely implement next.



Ijin Yu

```

# Convolution Based Implementation In the previous problem, you showed that the forward
pass of an SSM can be implemented using a convolution operation. In this problem, you will
implement the forward pass of an SSM using a convolution based approach. You can assume
that T is a power of 2. You will implement two functions - `make_conv_kernel(W, T)`: This
function will take the recurrent weight matrix $W$ and the number of time steps $T$ and return
the convolution kernel $K$. Given that T is a power of 2, you can implement this using a divide
and conquer based approach. - `conv_ssm_forward(W, U, b, x)`: This function will take weights
$W$, $U$, $b$ and input $x$ and return the hidden states $h$ across different time steps.
def make_conv_kernel(W, T): """ Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing the ssm forward pass using conv1d. Args: W: (H, H) weight matrix T: scalar
Returns: kernel_for_conv: (H, H, T) tensor """
#####
## # TODO: Implement the kernel here #
#####
##
#####
## # END OF YOUR CODE #
#####
## return kernel_for_convdef conv_ssm_forward(W, U, b, x): """ Convolution-based forward
pass for a batch of sequences. RNN update:  $h_{t+1} = W h_t + U x_t + b$  Args: W: (H, H) weight
matrix U: (H, D) input projection b: (H,) bias x: (N, T, D) input (batch=N, time steps=T, input
dim=D) Returns: h_all: (N, T, H) hidden states """ N, T, D = x.shape H = W.shape[0] s = x @ U.T
+ b s = s.permute(0, 2, 1) # Build the kernel with shape (H, H, T). kernel =
make_conv_kernel(W, T)
#####
## # TODO: Implement the convolution here #
#####
##
#####
## # END OF YOUR CODE #
#####
## return h_all

```

Claude Opus

Convolution-Based SSM Implementation

Let me implement both functions for the convolution-based forward pass.

Mathematical Background

From the recurrence $h_{t+1} = Wh_t + Ux_t + b$, we can unroll to get:

$$h_t = \sum_{i=0}^{t-1} W^{t-1-i} s_i$$

where $s_i = Ux_i + b$. This is a convolution between the sequence s and the kernel K where $K[:, :, t] = W^t$.

python

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
    implementing
    the ssm forward pass using conv1d.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """

    #####
    #####
    #                                     TODO: Implement the kernel here
    #

    #####
    #####

    H = W.shape[0]

    # Base case: T = 1, kernel is just W^0 = I
    if T == 1:
        return torch.eye(H, device=W.device,
dtype=W.dtype).unsqueeze(-1)

    # Divide and conquer approach
    # We need [W^0, W^1, W^2, ..., W^{T-1}]
```

```

# Recursively compute kernel for T/2
# This gives us [W^0, W^1, ..., W^{T/2-1}]
half_kernel = make_conv_kernel(W, T // 2)

# Compute W^{T/2}
W_half = torch.matrix_power(W, T // 2)

# The second half is W^{T/2} @ [W^0, W^1, ..., W^{T/2-1}]
# Which gives us [W^{T/2}, W^{T/2+1}, ..., W^{T-1}]
# half_kernel has shape (H, H, T/2)
# We need to multiply each slice by W_half on the left

# Reshape for batch matrix multiplication
# half_kernel: (H, H, T/2) -> (T/2, H, H)
half_kernel_permuted = half_kernel.permute(2, 0, 1)

# W_half @ each slice: (H, H) @ (T/2, H, H) -> use broadcasting
second_half_permuted = W_half @ half_kernel_permuted # (T/2, H,
H)

# Permute back to (H, H, T/2)
second_half = second_half_permuted.permute(1, 2, 0)

# Concatenate along the time dimension
kernel_for_conv = torch.cat([half_kernel, second_half], dim=-1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
return kernel_for_conv

```

```

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b # (N, T, H)

    s = s.permute(0, 2, 1) # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    #####
    #####
    #                                     TODO: Implement the convolution here
    #

    #####
    #####

    # We need causal convolution:  $h_t = \sum_{i=0}^{t-1} W^{t-1-i} s_i$ 
    # This means we need to flip the kernel and do conv1d with
    appropriate padding

```

```

# Pad s on the left with T-1 zeros for causal convolution
s_padded = torch.nn.functional.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Flip kernel along time dimension for convolution
# kernel[:, :, t] = W^t, we need W^{T-1-t} for conv1d
kernel_flipped = torch.flip(kernel, dims=[-1]) # (H, H, T)

# conv1d expects:
# input: (N, C_in, L) = (N, H, 2T-1)
# weight: (C_out, C_in, K) = (H, H, T)
# output: (N, C_out, L_out)

h_all = torch.nn.functional.conv1d(s_padded, kernel_flipped) #
(N, H, T)

# Permute back to (N, T, H)
h_all = h_all.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
return h_all

```

Explanation

make_conv_kernel(W, T)

Uses divide-and-conquer to build the kernel $K[:, :, t] = W^t K[:, :, t] = W^{T-t} K[:, :, t] = W^t$ for $t=0, 1, \dots, T-1$

1. **Base case:** When $T=1$, return the identity matrix $W^0 = I$
2. **Recursive step:**

- Compute the first half: $[W_0, W_1, \dots, W_{T/2-1}] [W^0, W^1, \dots, W^{T/2-1}]$ recursively
- Compute $W_{T/2} W^{T/2} W_{T/2}$ using matrix power
- The second half is $W_{T/2} \times [\text{first half}] = [W_{T/2}, W_{T/2+1}, \dots, W_{T-1}] W^{T/2} \times [\text{first half}] = [W^{T/2}, W^{T/2+1}, \dots, W^{T-1}] W_{T/2} \times [\text{first half}] = [W_{T/2}, W_{T/2+1}, \dots, W_{T-1}]$
- Concatenate both halves

This gives $O(\log T)$ recursive depth with $O(H^3)$ matrix multiplications at each level.

conv_ssm_forward(W, U, b, x)

1. Compute $s = Ux + bs = Ux + bs = Ux + b$ for all timesteps
2. Pad the input on the left for causal convolution (we only want past inputs to affect current output)
3. Flip the kernel (since convolution reverses the kernel, and we need $W_{t-1-i} W^{t-1-i}$ for the sum)
4. Apply `conv1d` to compute all hidden states in parallel



Ijin Yu

```
----- TypeError Traceback (most recent call
last) /tmp/ipython-input-1814800328.py in <cell line: 0>() 23 print("\nMax absolute difference:",
diff.item()) 24 ---> 25 sanity_check() 1 frames /tmp/ipython-input-1095872970.py in
unrolled_ssm_forward(W, U, b, x) 33 # h_t: (N, H), W: (H, H) -> h_t @ W.T gives (N, H) 34 # x[:,
t, :]: (N, D), U: (H, D) -> x[:, t, :] @ U.T gives (N, H) ---> 35 h_t = h_t @ W.T + x[:, t, :] @ U.T + b
36 h_all[:, t, :] = h_t 37 TypeError: unsupported operand type(s) for @: 'numpy.ndarray' and
'Tensor'
```

Claude Opus

The error shows that the code is mixing NumPy arrays with PyTorch tensors. The `unrolled_ssm_forward` function is creating NumPy arrays while the inputs are PyTorch tensors. Let me fix both implementations to use PyTorch consistently:

python

```

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
    Returns:
        h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
    """

    #####
    #####
    #                                TODO: Implement the recurrent pass here
    #

    #####
    #####

    N, T, D = x.shape
    H = W.shape[0]

    # Initialize output tensor to store all hidden states (use torch,
    not numpy)
    h_all = torch.zeros((N, T, H), device=x.device, dtype=x.dtype)

    # Initialize hidden state to zeros
    h_t = torch.zeros((N, H), device=x.device, dtype=x.dtype)

    # Unroll through time
    for t in range(T):
        #  $h_{t+1} = W @ h_t + U @ x_t + b$ 
        # For batched computation:

```

```

        # h_t: (N, H), W: (H, H) -> h_t @ W.T gives (N, H)
        # x[:, t, :]: (N, D), U: (H, D) -> x[:, t, :] @ U.T gives (N,
H)

        h_t = h_t @ W.T + x[:, t, :] @ U.T + b
        h_all[:, t, :] = h_t

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return h_all

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing
    the ssm forward pass using conv1d.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """

#####
#####
#                                     TODO: Implement the kernel here
#

#####
#####

```

```
H = W.shape[0]

# Base case: T = 1, kernel is just W^0 = I
if T == 1:
    return torch.eye(H, device=W.device,
dtype=W.dtype).unsqueeze(-1)

# Divide and conquer approach
# We need [W^0, W^1, W^2, ..., W^{T-1}]

# Recursively compute kernel for T/2
half_kernel = make_conv_kernel(W, T // 2)

# Compute W^{T/2}
W_half = torch.matrix_power(W, T // 2)

# The second half is W^{T/2} @ [W^0, W^1, ..., W^{T/2-1}]
# half_kernel: (H, H, T/2) -> (T/2, H, H)
half_kernel_permuted = half_kernel.permute(2, 0, 1)

# W_half @ each slice
second_half_permuted = W_half @ half_kernel_permuted # (T/2, H,
H)

# Permute back to (H, H, T/2)
second_half = second_half_permuted.permute(1, 2, 0)

# Concatenate along the time dimension
kernel_for_conv = torch.cat([half_kernel, second_half], dim=-1)

#####
#####
#
#
```

```

#####
#####
    return kernel_for_conv

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b # (N, T, H)

    s = s.permute(0, 2, 1) # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

#####
#####
    #                                TODO: Implement the convolution here
#

```

```
#####
#####

# Pad s on the left for causal convolution
s_padded = torch.nn.functional.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Flip kernel along time dimension
kernel_flipped = torch.flip(kernel, dims=[-1]) # (H, H, T)

# conv1d: input (N, H, 2T-1), weight (H, H, T) -> output (N, H, T)
h_all = torch.nn.functional.conv1d(s_padded, kernel_flipped) #
(N, H, T)

# Permute back to (N, T, H)
h_all = h_all.permute(0, 2, 1)

#####
#####

#                                END OF YOUR CODE

#

#####
#####

return h_all
```

The key fix was replacing `np.zeros` with `torch.zeros` and ensuring the tensors are created on the same device and with the same dtype as the input tensors. This maintains consistency throughout the computation.



Ijin Yu

Question 1 What maximum absolute difference do you observe between the outputs of the two implementations for the following inputs? Question 2 What is the number of operations being performed in the recurrence based implementation of the SSM forward pass? Question 3 What is the number of operations being performed in the convolution based implementation of the SSM forward pass? Question 4 Compare the trade-off if any between the two implementations based on your answers above.\

Claude Opus

Analysis of SSM Implementations

Question 1: Maximum Absolute Difference

The maximum absolute difference between the two implementations should be on the order of **1e-6 to 1e-5** (for float32) or **1e-14 to 1e-15** (for float64).

This small difference arises from floating-point numerical precision differences between the two mathematically equivalent approaches. The convolution-based method accumulates errors differently than the sequential recurrence.

To verify, you can run:

```
def sanity_check():
    torch.manual_seed(42)
    N, T, D, H = 2, 16, 8, 4

    W = torch.randn(H, H) * 0.1 # Scale down for stability
    U = torch.randn(H, D)
    b = torch.randn(H)
    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = torch.max(torch.abs(h_unrolled - h_conv))
    print(f"Max absolute difference: {diff.item()}")

sanity_check()
```

Question 2: Operations in Recurrence-Based Implementation

For each timestep, we compute: $h_{t+1} = W h_t + U x_t + b h_{t+1} = W h_t + U x_t + b$

Per timestep operations:

- $W h_t$: Matrix-vector multiplication $(H \times H) \cdot (H) \rightarrow O(H^2)$ operations per batch element
- $U x_t$: Matrix-vector multiplication $(H \times D) \cdot (D) \rightarrow O(HD)$ operations per batch element
- Adding b and the two terms: $O(H)$ operations

Total operations: $O(N \cdot T \cdot (H^2 + HD))$

Or equivalently: $O(NTH(H+D))$

The key characteristic is that **T appears linearly** and the computation is **sequential** (cannot be parallelized across time).

Question 3: Operations in Convolution-Based Implementation

Step 1: Compute $s = Ux + bs = Ux + b$

- For all timesteps: $O(N \cdot T \cdot H \cdot D)$

Step 2: Build the convolution kernel using divide-and-conquer

- Depth of recursion: $\log_2(T)$
- At each level: one matrix power computation $O(H^3)$ and matrix multiplications with the half-kernel
- Total: $O(H^3 \log T)$ for kernel construction

Step 3: 1D Convolution

- Input: (N, H, T) , Kernel: (H, H, T)
- Each output element requires summing over $H \times H$ multiplications
- Total output elements: $N \times H \times T$

- Convolution cost: $O(N \cdot H^2 \cdot T^2)$ for direct convolution

If using **FFT-based convolution**:

- FFT of input and kernel: $O(T \log T)$
- Total: $O(N \cdot H^2 \cdot T \log T)$

Total operations (direct convolution): $O(NTH \cdot D + H^3 \log T + NH^2 T^2)$

Total operations (FFT-based): $O(NTH \cdot D + H^3 \log T + NH^2 T \log T)$

Question 4: Trade-off Comparison

Aspect	Recurrence-Based	Convolution-Based
Time Complexity	$O(NTH(H+D))$	$O(NH^2 T^2)$ or $O(NH^2 T \log T)$ with FFT
Parallelism	Sequential in TTT (cannot parallelize across time)	Fully parallelizable
Memory	$O(H)$ intermediate state	$O(H^2 T)$ for kernel storage
Hardware Utilization	Poor GPU utilization due to sequential nature	Excellent GPU utilization

Key Trade-offs:

1. **Parallelism vs. Complexity:** The recurrence has lower theoretical complexity but is inherently sequential. The convolution has higher complexity but is embarrassingly parallel, making it much faster on GPUs in practice.
2. **Memory:** Convolution requires storing the full kernel of size (H, H, T) , while recurrence only needs the current hidden state.
3. **Sequence Length Scaling:** For very long sequences, convolution with FFT scales as $O(T \log T)$ per position versus $O(T)$ total for recurrence, but the parallelism advantage often outweighs this.

4. **Practical Performance:** On modern GPUs, the convolution-based approach is typically **significantly faster** despite higher FLOP count because it avoids the sequential bottleneck of the recurrence. The recurrence cannot utilize the thousands of GPU cores effectively since each step depends on the previous one.



Ijin Yu

remember we r working with cpu implementation right now. also, ignore d and n but only focus on $O()$ in terms of T and H.

Claude Opus

Analysis of SSM Implementations (CPU, focusing on T and H)

Question 1: Maximum Absolute Difference

The maximum absolute difference between the two implementations should be on the order of **1e-6 to 1e-5** (for float32) or **1e-14 to 1e-15** (for float64).

This small difference arises from floating-point numerical precision differences between the two mathematically equivalent approaches.

Question 2: Operations in Recurrence-Based Implementation

For each timestep, we compute: $h_{t+1} = W h_t + U x_t + b_{h_{t+1}} = W h_t + U x_t + b$

Per timestep operations:

- $W_{ht}W_{h_t}W_{ht}$: Matrix-vector multiplication $\rightarrow O(H^2)O(H^2)O(H^2)$
- Other operations: $O(H)O(H)O(H)$

Total over T timesteps: $O(T \cdot H^2) \boxed{O(T \cdot H^2)} O(T \cdot H^2)$

The computation is **sequential** — each step depends on the previous hidden state.

Question 3: Operations in Convolution-Based Implementation

Step 1: Build the convolution kernel using divide-and-conquer

- Recursion depth: $\log_2(T)$
- At each level: matrix power and multiplications with half-kernel slices
- Total: $O(H^3 \log T) O(H^3 \log T) O(H^3 \log T)$

Step 2: 1D Convolution

- Kernel has TTT slices, each of size $(H, H)(H, H)(H, H)$
- For each of TTT output positions, we sum over TTT kernel positions with $H \times H \times H$ matrix operations
- Direct convolution: $O(T^2 \cdot H^2) O(T^2 \cdot H^2) O(T^2 \cdot H^2)$

Total operations: $O(H^3 \log T + T^2 H^2) \boxed{O(H^3 \log T + T^2 H^2)} O(H^3 \log T + T^2 H^2)$

Question 4: Trade-off Comparison (CPU)

Aspect	Recurrence-Based	Convolution-Based
Time Complexity	$O(TH^2) O(TH^2) O(TH^2)$	$O(H^3 \log T + T^2 H^2) O(H^3 \log T + T^2 H^2) O(H^3 \log T + T^2 H^2)$
Parallelism	Sequential in TTT	Parallelizable
Memory	$O(H)O(H)O(H)$ for state	$O(H^2 T) O(H^2 T) O(H^2 T)$ for kernel

Key Trade-offs:

1. **Computational Cost:** The recurrence-based approach is asymptotically more efficient with $O(TH^2)O(TH^2)O(TH^2)$ versus $O(T^2H^2)O(T^2H^2)O(T^2H^2)$ for convolution. For long sequences, recurrence wins on raw FLOP count.
2. **Parallelism on CPU:** Even on CPU, convolution operations can leverage optimized BLAS libraries and SIMD instructions more effectively. However, the T^2H^2 factor in convolution makes it slower for large TTT.
3. **Memory:** Convolution requires storing the full kernel $O(H^2T)O(H^2T)O(H^2T)$, while recurrence only needs $O(H)O(H)O(H)$ for the current state.
4. **Practical Recommendation:** On CPU with large TTT, the **recurrence-based approach is generally faster** due to the $O(T)O(T)O(T)$ vs $O(T^2)O(T^2)O(T^2)$ scaling, despite being sequential.



Ijin Yu

actually, convoluion based runtime is wrong. The dominant cost comes from the `make_conv_kernel` function. To compute the kernel powers $[W^0, W^1, \dots, W^{T-1}]$ using the divide-and-conquer approach, what is the runtime?

Claude Opus

Corrected Analysis of Convolution-Based Implementation

Question 3: Operations in Convolution-Based Implementation

Let me re-analyze the `make_conv_kernel` function carefully.

Divide-and-Conquer Recurrence:

At each level, we:

1. Recursively compute the first half: $[W_0, W_1, \dots, W_{T/2-1}] [W^0, W^1, \dots, W^{T/2-1}]$
2. Compute $W_{T/2} W^{T/2} W_{T/2}$ using matrix power: $O(H^3 \log T) O(H^3 \log T) O(H^3 \log T)$
3. Multiply $W_{T/2} W^{T/2} W_{T/2}$ with each of the $T/2$ slices to get the second half: $O(T^2 \cdot H^3) O(\frac{T}{2} \cdot H^3) O(2T \cdot H^3)$

Recurrence relation: $C(T) = C(T/2) + O(H^3 \log T) + O(T^2 \cdot H^3) C(T) = C(T/2) + O(H^3 \log T) + O(\frac{T}{2} \cdot H^3) C(T) = C(T/2) + O(H^3 \log T) + O(2T \cdot H^3)$

The dominant term at each level is the $O(T^2 \cdot H^3) O(\frac{T}{2} \cdot H^3) O(2T \cdot H^3)$ matrix multiplications.

Expanding the recurrence:

- Level 0: $T^2 \cdot H^3 \frac{T}{2} \cdot H^3$
- Level 1: $T^4 \cdot H^3 \frac{T}{4} \cdot H^3$
- Level 2: $T^8 \cdot H^3 \frac{T}{8} \cdot H^3$
- ...
- Level $\log T - 1$: $1 \cdot H^3$

Total: $H^3 \cdot (T^2 + T^4 + T^8 + \dots + 1) = H^3 \cdot (T - 1) = O(T H^3) H^3 \cdot \left(\frac{T}{2} + \frac{T}{4} + \frac{T}{8} + \dots + 1 \right) = H^3 \cdot (T - 1) = O(T H^3) H^3 \cdot (2T + 4T + 8T + \dots + 1) = H^3 \cdot (T - 1) = O(T H^3)$

Convolution-based total runtime: $O(T H^3) \boxed{O(T H^3)} O(T H^3)$

Question 4: Revised Trade-off Comparison (CPU)

Aspect	Recurrence-Based	Convolution-Based
Time Complexity	$O(T H^2) O(T H^2) O(T H^2)$	$O(T H^3) O(T H^3) O(T H^3)$

Conclusion: The recurrence-based approach is more efficient by a factor of H^3 . On CPU, the sequential recurrence with $O(T H^2) O(T H^2) O(T H^2)$ outperforms the convolution approach with $O(T H^3) O(T H^3) O(T H^3)$, especially as the hidden dimension grows.



Ijin Yu

Introduction In this notebook, we'll implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage. You will need GPU for this notebook which can be enabled via changing the runtime. You can copy your solutions from the

q_coding_ssm_forward_cpu notebook for the first part of the notebook.
SSM Update Rule We consider an RNN described by the update: $h_{t+1} = W h_t + U x_t + b$ for $t = 0, 1, \dots, T - 1$. The variables are: - $h_t \in \mathbb{R}^H$, the hidden state at time t . - $x_t \in \mathbb{R}^{N \times D}$, the input at time t . - $W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix. - $U \in \mathbb{R}^{H \times D}$, the input projection matrix. - $b \in \mathbb{R}^H$, the bias vector. N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H .
def unrolled_ssm_forward(W, U, b, x): """ Unroll the linear RNN in time: $h_{t+1} = W h_t + U x_t + b$ with initial $h_0 = 0$. Args: W: (H, H) weight matrix U: (H, D) input projection b: (H,) bias x: (N, T, D) input sequence over T steps Returns: h_all: (N, T, H) hidden states for $t=1..T$ ($h_all[t]$ corresponds to h_{t+1} in the usual notation). """

#####

TODO: Implement the recurrent pass here

#####

##

#####

END OF YOUR CODE

#####

return h_all# Convolution Based Implementation In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach.

You can assume that T is a power of 2. You will implement two functions -

`make\_conv\_kernel(W, T)`: This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K . Given that T is a power of 2, you can implement this using a divide and conquer based approach. - `conv\_ssm\_forward(W, U, b, x)`: This function will take weights W , U , b and input x and return the hidden states h across different time steps.

def make_conv_kernel(W, T): """ Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing the ssm forward pass using conv1d. Args: W: (H, H) weight matrix T: scalar Returns: kernel_for_conv: (H, H, T) tensor """

#####

TODO: Implement the kernel here

#####

##

```
#####
## # END OF YOUR CODE #
#####
## return kernel_for_convdef conv_ssm_forward(W, U, b, x): """ Convolution-based forward
pass for a batch of sequences. RNN update:  $h_{t+1} = W h_t + U x_t + b$  Args: W: (H, H) weight
matrix U: (H, D) input projection b: (H,) bias x: (N, T, D) input (batch=N, time steps=T, input
dim=D) Returns: h_all: (N, T, H) hidden states """ N, T, D = x.shape H = W.shape[0] s = x @ U.T
+ b s = s.permute(0, 2, 1) # Build the kernel with shape (H, H, T). kernel =
make_conv_kernel(W, T)
#####
## # TODO: Implement the convolution here #
#####
##
#####
## # END OF YOUR CODE #
#####
## return h_all
```

Claude Opus

SSM Forward Pass Implementation (GPU)

Here are the implementations for both approaches:

python

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
    Returns:
        h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
    """
```

```
#####
#####
#                               TODO: Implement the recurrent pass here
#

#####
#####

    N, T, D = x.shape
    H = W.shape[0]

    # Initialize output tensor to store all hidden states
    h_all = torch.zeros((N, T, H), device=x.device, dtype=x.dtype)

    # Initialize hidden state to zeros
    h_t = torch.zeros((N, H), device=x.device, dtype=x.dtype)

    # Unroll through time
    for t in range(T):
        h_t = h_t @ W.T + x[:, t, :] @ U.T + b
        h_all[:, t, :] = h_t

#####
#####
#                               END OF YOUR CODE
#

#####
#####

    return h_all

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
    implementing
```


the ssm forward pass using conv1d.

Args:

W: (H, H) weight matrix

T: scalar

Returns:

kernel_for_conv: (H, H, T) tensor

"""

```
#####  
#####
```

```
    #                                TODO: Implement the kernel here  
#
```

```
#####  
#####
```

```
H = W.shape[0]
```

```
# Base case: T = 1, kernel is just  $W^0 = I$ 
```

```
if T == 1:
```

```
    return torch.eye(H, device=W.device,  
dtype=W.dtype).unsqueeze(-1)
```

```
# Recursively compute kernel for T/2
```

```
half_kernel = make_conv_kernel(W, T // 2)
```

```
# Compute  $W^{\{T/2\}}$ 
```

```
W_half = torch.matrix_power(W, T // 2)
```

```
# Second half:  $W^{\{T/2\}} @ [W^0, W^1, \dots, W^{\{T/2-1\}}]$ 
```

```
# half_kernel: (H, H, T/2) -> permute to (T/2, H, H) for batch  
matmul
```

```
half_kernel_permuted = half_kernel.permute(2, 0, 1)
```

```
second_half_permuted = W_half @ half_kernel_permuted
```

```
second_half = second_half_permuted.permute(1, 2, 0)
```

```

    # Concatenate along the time dimension
    kernel_for_conv = torch.cat([half_kernel, second_half], dim=-1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return kernel_for_conv

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b # (N, T, H)

    s = s.permute(0, 2, 1) # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

```

```
#####
#####
#                                     TODO: Implement the convolution here
#

#####
#####

# Pad s on the left for causal convolution
s_padded = torch.nn.functional.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Flip kernel along time dimension for convolution
kernel_flipped = torch.flip(kernel, dims=[-1]) # (H, H, T)

# conv1d: input (N, H, 2T-1), weight (H, H, T) -> output (N, H, T)
h_all = torch.nn.functional.conv1d(s_padded, kernel_flipped) #
(N, H, T)

# Permute back to (N, T, H)
h_all = h_all.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
return h_all
```

Complexity Comparison (GPU Context)

Aspect	Recurrence-Based	Convolution-Based
--------	------------------	-------------------

Time Complexity	$O(TH^2)$ $O(TH^2)$	$O(TH^3)$ $O(TH^3)$
Sequential Steps	$O(T)$ $O(T)$	$O(\log T)$ $O(\log T)$
Parallelism	Poor (sequential in T)	Excellent

On GPU: Despite higher FLOP count, convolution is often **faster** because GPUs excel at parallel operations. The recurrence has TTT sequential steps that cannot be parallelized, severely underutilizing GPU cores. The convolution kernel construction has only $O(\log T)$ sequential depth, and the convolution itself is highly parallel.



Ijin Yu

correct. but can you think about what will happen as H increases?

Claude Opus

Effect of Increasing H on GPU Performance

As HHH increases, the trade-off between the two approaches shifts:

Complexity Recap

Approach	Time Complexity	Sequential Steps
Recurrence	$O(TH^2)$ $O(TH^2)$	$O(T)$ $O(T)$
Convolution	$O(TH^3)$ $O(TH^3)$	$O(\log T)$ $O(\log T)$

What Happens as H Grows?

Recurrence-based approach:

- Each step performs $H \times HH \times HH \times H$ matrix-vector multiplication: $O(H^2)O(H^2)O(H^2)$
- But each step is **sequential** — GPU cores sit idle waiting for previous step
- However, as HHH increases, each individual step becomes more parallelizable (larger matrix operations utilize GPU better)

Convolution-based approach:

- Kernel construction involves TTT matrix multiplications of size $H \times HH \times HH \times H$: $O(TH^3)O(TH^3)O(TH^3)$
- The extra factor of HHH compared to recurrence becomes increasingly significant

Crossover Point

For **small H**:

- Recurrence underutilizes GPU (small matrix ops, many sequential steps)
- Convolution wins due to parallelism despite higher FLOP count

For **large H**:

- Recurrence steps become large enough to saturate GPU cores
- The $O(H^3)O(H^3)O(H^3)$ vs $O(H^2)O(H^2)O(H^2)$ gap becomes dominant
- **Recurrence starts to win** because the extra HHH factor in convolution is too costly

Convolution Cost / Recurrence Cost = $\frac{O(TH^3)O(TH^3)O(TH^3)}{O(H^2)O(H^2)O(H^2)} = O(H) \frac{\text{Convolution Cost}}{\text{Recurrence Cost}} = \frac{O(TH^3)}{O(TH^2)} = O(H)$
Convolution Cost = $O(TH^2)O(TH^3) = O(H)$

As HHH increases, convolution becomes HHH times more expensive relatively.

Conclusion

There exists a **crossover point** where:

- Below some threshold HHH : convolution is faster (parallelism advantage dominates)
- Above that threshold HHH : recurrence is faster (the $H^3H^3H^3$ vs $H^2H^2H^2$ cost dominates)

This is a classic **parallelism vs. computational complexity** trade-off. The optimal choice depends on the specific values of TTT and HHH , as well as the GPU architecture.



Ijin Yu

Introducing structure: Diagonal Weight Matrices We can optimize the convolution implementation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging QAg depthwise convolutions for implementing the forward operation.

```
def make_diag_depthwise_kernel(W, T): """ Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matrix. Args: W: (H, H) diagonal matrix T: (int) number of time steps Return: kernel of shape (H, 1, T), which can be used in depthwise convolution """
```

```
#####  
## # TODO: Implement the kernel here #
```

```
#####  
##
```

```
#####  
## # END OF YOUR CODE #
```

```
#####  
## return kerneldef diag_conv_ssm_forward(W, U, b, x): """ Convolution-based forward pass for an RNN with W RNN update:  $h_{t+1} = W h_t + U x_t + b$  but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ . Args: W: (H, H) [diagonal in practice] U: (H, D) b: (H,) x: (N, T, D) => N = batch, T = time steps, D = input dim Returns: h_all: (N, T, H) """ N, T, D = x.shape H = W.shape[0] s = x @ U.T + b s = s.permute(0, 2, 1) # (N,H,T) # Build kernel (H,1,T) for depthwise conv kernel = make_diag_depthwise_kernel(W, T)
```

```
#####  
## # TODO: Implement the convolution here # # Hint: Use `groups` argument #
```

```
#####  
##
```

```
#####  
## # END OF YOUR CODE #
```

```
#####  
## return h_all # Optimizing the recurrent implementation Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the  $W$  matrix.
```

```
def diag_unrolled_ssm_forward(W, U, b, x): """ Forward pass for:  $h_{t+1} = W h_t + U x_t + b$  but W is diagonal, i.e.  $W = \text{diag}(\text{diagW})$ . Args: W: (H, H) diagonal matrix U: (H, D) b: (H,) x: (N, T, D) => batch=N, time steps=T, input dim=D Returns: h_all: (N, T, H) """
```

```
#####  
## # TODO: Implement the optimized recurrent pass here #
```

```
#####  
##
```

```
#####
## # END OF YOUR CODE #
#####
## return h_all
```

Claude Opus

Diagonal Weight Matrix Optimizations

When WWW is diagonal, we can significantly optimize both implementations. Instead of full matrix multiplications, we only need element-wise operations.

Key Insight

If $W = \text{diag}(w_1, w_2, \dots, w_H)$, then:
 $h_{t+1}[i] = w_i \cdot h_t[i] + (Ux_t + b)[i]$

Each hidden dimension evolves independently, allowing for depthwise (channel-wise) operations.

For the kernel: $W_t = \text{diag}(w_{1t}, w_{2t}, \dots, w_{Ht})$,
 $W_t^t = \text{diag}(w_{1^t}, w_{2^t}, \dots, w_{H^t})$

```
python
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is
    a diagonal matrix.

    Args:
        W: (H, H) diagonal matrix
        T: (int) number of time steps

    Return:
        kernel of shape (H, 1, T), which can be used in depthwise
    convolution
    """
```

```

#####
#####
#                                     TODO: Implement the kernel here
#

#####
#####

    H = W.shape[0]

    # Extract diagonal elements
    diag_w = torch.diag(W) # (H,)

    # Compute powers: [w^0, w^1, w^2, ..., w^{T-1}] for each diagonal
    element
    # Create exponents [0, 1, 2, ..., T-1]
    exponents = torch.arange(T, device=W.device, dtype=W.dtype) #
    (T,)

    # diag_w: (H,) -> (H, 1)
    # exponents: (T,) -> (1, T)
    # Result: (H, T) where kernel[i, t] = w_i^t
    kernel = diag_w.unsqueeze(1) ** exponents.unsqueeze(0) # (H, T)

    # Reshape to (H, 1, T) for depthwise conv
    kernel = kernel.unsqueeze(1) # (H, 1, T)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return kernel

```



```

def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

    Args:
        W: (H, H) [diagonal in practice]
        U: (H, D)
        b: (H,)
        x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
        h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1) # (N, H, T)

    # Build kernel (H, 1, T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

#####
#####
#                                     TODO: Implement the convolution here
#
#                                     Hint: Use `groups` argument
#
#####
#####

```

```

# Pad s on the left for causal convolution
s_padded = torch.nn.functional.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Flip kernel along time dimension
kernel_flipped = torch.flip(kernel, dims=[-1]) # (H, 1, T)

# Depthwise conv1d with groups=H (each channel convolved
independently)
# Input: (N, H, 2T-1), Kernel: (H, 1, T), groups=H -> Output: (N,
H, T)
h_all = torch.nn.functional.conv1d(s_padded, kernel_flipped,
groups=H)

# Permute back to (N, T, H)
h_all = h_all.permute(0, 2, 1)

#####
#####
#                                     END OF YOUR CODE
#

#####
#####
    return h_all

def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
         $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, i.e.  $W = \text{diag}(\text{diagW})$ .

    Args:
        W: (H, H) diagonal matrix
        U: (H, D)
        b: (H,)

```

```

    x: (N, T, D) => batch=N, time steps=T, input dim=D

Returns:
    h_all: (N, T, H)
    """

#####
#####
#           TODO: Implement the optimized recurrent pass here
#

#####
#####

N, T, D = x.shape
H = W.shape[0]

# Extract diagonal elements for element-wise multiplication
diag_w = torch.diag(W) # (H,)

# Initialize output tensor
h_all = torch.zeros((N, T, H), device=x.device, dtype=x.dtype)

# Initialize hidden state
h_t = torch.zeros((N, H), device=x.device, dtype=x.dtype)

# Unroll through time with element-wise operations
for t in range(T):
    # Element-wise multiplication instead of matrix multiplication
    # h_t * diag_w: (N, H) * (H,) -> (N, H) via broadcasting
    h_t = h_t * diag_w + x[:, t, :] @ U.T + b
    h_all[:, t, :] = h_t

#####
#####
#           END OF YOUR CODE
#

```

```
#####  
#####  
    return h_all
```

Complexity Comparison with Diagonal W

Approach	Original (Full W)	Diagonal W
Recurrence	$O(TH^2)O(TH^2)O(TH^2)$	$O(TH)O(TH)O(TH)$
Convolution (kernel)	$O(TH^3)O(TH^3)O(TH^3)$	$O(TH)O(TH)O(TH)$
Convolution (conv op)	$O(T^2H^2)O(T^2H^2)O(T^2H^2)$ or $O(TH^2)O(TH^2)O(TH^2)$	$O(T^2H)O(T^2H)O(T^2H)$ or $O(TH)O(TH)O(TH)$ depthwise

The diagonal constraint reduces complexity by a factor of HHH for both approaches, making them much more scalable for large hidden dimensions.